Lightweight Trap-Based Free-Rider Detection and Mitigation in Federated Learning

Aashnan Rahman
Department of Computer Science and Engineering
Islamic University of Technology
MSc Thesis

Abstract—Federated learning relies on participants performing useful local optimization, but free riders can consume the shared model while submitting unchanged or fabricated responses. Detecting such behavior is difficult when honest clients have heterogeneous data and naturally different update statistics. This paper investigates a server-side challenge protocol that compares responses to undisclosed perturbations of the same model. Robust update-norm and tensor-profile measurements first nominate anomalous clients; separate geometric separation tests and repeated fresh evidence authorize removal. Component-specific memory retains strong evidence across ambiguous cycles, while bounded quarantine prevents weak suspicion from causing indefinite exclusion. The method requires no auxiliary detector training, client-reported loss, or attack-specific history reconstruction. We evaluate 32 completed 100-round runs covering four fabrication strategies, MNIST and CIFAR-10, IID and Dirichlet non-IID data, and 30% and 40% free riders. All 1120 free-rider client-cases are removed, with no permanent removal among 2080 honest client-cases. Complete removal occurs at round 30 except for non-IID CIFAR-10 history averaging, which requires rounds 60 and 80. However, 11 honest client-cases experience temporary quarantine, and substantial weak candidate populations remain. The findings support the approach under the evaluated setting, not a universal detection guarantee: the study uses one development seed and does not evaluate adaptive adversaries or secure aggregation.

Index Terms—federated learning, free-rider detection, hidden challenges, robust statistics, non-IID data, client auditing

I. Introduction

II. Introduction

Picture a few thousand phones running the same keyboard app. Each one quietly learns from what its owner types – new words, favorite phrases, the odd misspelling a particular person always makes – and none of that typing history ever needs to leave the device. The app still gets better for everyone, because every phone periodically sends back a small update describing what it learned, and a server blends these updates into an improved model that is then sent back out. This is the basic idea behind federated learning (FL): instead of gathering everyone's data in one place to train a model, the model is sent to where the data already lives [1]. A server distributes the current model, each participant trains it briefly on local data, and the server combines the returned updates into the next version of the model.

The arrangement only works if participants actually do the training they are asked to do. Suppose one phone in that keyboard example never really trains its copy of the model – maybe a battery-saver setting quietly skips the background job, or maybe the app is simply configured to hand back whatever it was given. That phone still downloads every improved model the server produces. The server, with no way to look inside any device, has little reason to treat its response differently from an honest one. This is free riding: taking part in name only, collecting the benefit of a collaboration that other people are paying for [2], [3]. It is a different problem from an attacker trying to steer the model's predictions [4]; a free rider usually has nothing to gain from making the model worse. It just has nothing to contribute either.

The cost is easy to state and easy to underestimate. Had that phone trained honestly, its owner's particular habits and vocabulary could have added something the rest of the population did not already have. Multiply that by hundreds of silent participants, and the resulting model is simply worse than it could have been, even though every round of training appeared to go smoothly. Catching the problem is harder than it sounds, though, because a free rider's update does not have to look broken. Honest participants naturally differ from one another – someone with little data, an older device, or an unusual vocabulary produces an update that looks atypical while training perfectly honestly. A server that flags anything unusual ends up punishing exactly the diversity that makes federated learning worth using in the first place, while one that is too forgiving lets a free rider blend in with the honest participants it is hiding among.

This paper asks whether occasional, targeted audits can tell the difference, without the server ever inspecting a participant's data or simply trusting what a participant says about its own effort. Instead of always sending out the true shared model, the server occasionally sends a participant a slightly altered version and observes what comes back. A client that is actually training responds in a way shaped by that alteration; a client that is not has a harder time producing a response that fits an alteration it was never told about. We turn this difference into a small set of lightweight checks run entirely at the server, so that catching free riders does not require training a detector of its own or asking clients to report on themselves.

The main contributions of this work are as follows:

- We present a server-side method for identifying participants that are not genuinely training, based only on the models they return – with no access to private data, no reliance on a client's own report of its effort, and no separate detection model to train or maintain.
- We design a confirmation process so that a single odd-looking update is never enough to remove an honest client: participants under suspicion are checked again before anything permanent happens, and those that look unusual but keep training normally are cleared rather than punished indefinitely.
- We evaluate the approach against four different free-rider behaviors, ranging from doing nothing at all to carefully imitating what an honest update should look like, on MNIST and CIFAR-10, under both uniform and skewed data distributions, and at different proportions of free riders in the population.

The remainder of the paper reviews related work, defines the threat model and the free-rider behaviors evaluated, describes the auditing method in detail, and reports the resulting detection performance, computational cost, and limitations.

III. Literature Review

A. Free-Rider Attacks and Observable Update Geometry

Lin et al. introduced fabrication attacks in FL and proposed STD-DAGMM for detecting anomalous high-dimensional model updates [2]. Their work establishes that a client can participate without a local dataset, so receipt of a parameter vector cannot itself be treated as evidence of useful training. It also motivates examining update statistics rather than relying on an explicit declaration of participation.

Fraboni et al. analyzed free riding under iterative model aggregation and studied stochastic disguises that follow the evolution of honest models [3]. Zhu et al. subsequently constructed attacks informed by the geometric relationships of local updates, including carefully scaled noise [5]. Together, these works motivate testing more than unchanged-model replay. They also expose a limitation shared by feature-based defenses: an attacker that can reproduce the observed features may become indistinguishable from an honest client to that detector. Our four simulator behaviors cover different fabrication mechanisms, but are not claimed to reproduce every published advanced attack.

B. Statistical, Learned, and Contribution-Based Defenses

STD-DAGMM learns a representation and density model over submitted updates [2]. This can capture patterns beyond a single scalar threshold, but introduces an auxiliary model and its training procedure. In contrast, the present method retains compact norm/profile measurements and counters. This architectural difference is not an experimentally established runtime advantage: we do not perform a matched baseline benchmark here.

Hybrid methods also combine update-anomaly models with contribution evaluation. FRAD addresses free-rider detection in AIoT [6]; Recasens et al. characterize its detector as DAGMM augmented by contribution evaluation [7]. We do not reproduce FRAD or claim numerical superiority over it. Instead, the present study examines the tradeoff from replacing auxiliary detector training with controlled model challenges and geometric confirmation.

Parameter auditing offers another route. PASS combines contribution evaluation with privacy-oriented parameter processing and studies both anonymous and selfish free riders [8]. Its evaluation includes settings with an adversarial majority, whereas the present protocol assumes an honest majority. PASS's authors also discuss the communication cost of frequent auditing. This is an important distinction: avoiding an auxiliary learned detector does not eliminate the cost of challenge responses or withheld honest updates.

Statistical and contribution-based judgments also depend on the reference population. An honest but atypical participant can be a low-ranked contributor without being a free rider. Rather than equating contribution ranking with misconduct, our design uses population statistics for nomination and adds a separate repeated-evidence gate before permanent removal. Its geometric gate remains a heuristic, not a calibrated hypothesis test.

C. Challenge-Based Detection and Privacy Requirements

FRIDA uses membership- and property-inference mechanisms to seek evidence of genuine client training [7]. Its membership-based variants use canary examples, while its property-based variants examine inferred dataset characteristics. Our method instead perturbs the model input and evaluates update geometry, without distributing an additional training dataset. This reduces that particular dependency, but also yields weaker evidence: geometric separation does not demonstrate that a private training example was used.

Privacy constraints apply to both approaches. Secure aggregation is designed to reveal an aggregate without exposing individual client updates [9]. The present detector requires attributable responses, so it cannot be inserted unchanged into an ordinary secure-aggregation pipeline. A secure computation of audit features, a trusted execution environment, or a separate attributable audit channel would be a different design with additional assumptions.

The resulting research question is deliberately narrower than universal verification of training: can controlled model challenges, compact statistics, and conservative repeated confirmation distinguish the evaluated free-rider behaviors from heterogeneous honest training while limiting permanent and temporary harm to honest participants?

IV. Threat Model

A. Participants and Server Knowledge

We consider a coordinating server and N stable, attributable clients. Every active client participates in each

round until permanently removed. Honest clients execute the prescribed local optimizer on their local data, including when the received state is a trap. The server may send different model states to different clients and associate each returned state with its recipient. Communication integrity and stable identity are assumptions, not mechanisms implemented by the detector.

For client i in round t, the observable quantities are the sent and returned states, $w_{t,i}^{\text{send}}$ and $w_{t,i}^{\text{ret}}$, with

$$\Delta_{t,i} = w_{t,i}^{\text{ret}} - w_{t,i}^{\text{send}}, \qquad r_{t,i} = \|\Delta_{t,i}\|_2. \tag{1}$$

The server also knows public IDs and its own challenge schedule. It does not use attacker type, attacker identities, attack prevalence, private loss, dataset size, training logs, or reported resource use to decide whom to flag. The simulator necessarily knows attack identities to construct responses and compute evaluation metrics; these labels are attached to exported observations after the detector has made its decisions.

B. Adversary Capability and Evaluation Scope

The evaluated free riders perform no local optimization. Each can use its own current and previously received model states and local randomness. No trap-status flag is sent. The four implementations do not attempt to identify traps, share responses, change identity, or adapt to the detector's decisions. We do not assume that a real attacker cannot inspect its received state or know the protocol. In particular, an undisclosed trap flag is not a proof of trap indistinguishability.

The study assumes an honest majority and evaluates 30% and 40% free riders. Each run uses one fixed attack behavior for its entire attacker population. Collusion, Sybil identities, partial/on-off training, mixed strategies, dropouts, late arrivals, and defense-aware evasion are outside the evaluated grid.

C. Evaluated Free-Rider Behaviors

FR1: current-model sendback. The client returns the received model unchanged,

$$\widehat{w}_{t,i} = w_{t,i}^{\text{send}}. \tag{2}$$

Its delta is exactly zero, including on trap rounds. This differs from the previous-model playback definition used in the earlier v2/v8 manuscript. The current detector separately retains an exact previous-sendback check, but the present FR1 experiment evaluates current sendback.

FR2: bounded random fabrication. For every floating coordinate, the client submits

$$\widehat{w}_{t,i} = w_{t,i}^{\text{send}} + \epsilon_t, \quad (\epsilon_t)_j \sim \mathcal{U}(-10^{-3}, 10^{-3}). \tag{3}$$

The nonzero response avoids direct equality but contains no trained contribution. The detector does not receive the noise distribution or its amplitude.

FR3: received-history averaging. The client returns the mean of its most recent received states,

$$\widehat{w}_{t,i} = \frac{1}{|\mathcal{H}_{t,i}|} \sum_{w \in \mathcal{H}_{t,i}} w, \qquad |\mathcal{H}_{t,i}| \le 5. \tag{4}$$

The current state is included, and every received trap enters the same history. The detector does not reconstruct or compare against this five-model average.

FR4: received-trajectory imitation. The client computes recent received-state differences d_s and submits

$$d_s = w_{s,i}^{\text{send}} - w_{s-1,i}^{\text{send}}, \qquad \widehat{w}_{t,i} = w_{t,i}^{\text{send}} + \overline{d}_t + \eta_t, \tag{5}$$

where $\overline{d}_t$ averages up to five differences. Gaussian noise is rescaled to 10% of $\max(\|\overline{d}_t\|_2, 10^{-6})$, using a 10^{-6} numerical floor also in its normalization. Before a difference exists, noise has coordinate standard deviation 10^{-4}. These are attacker simulation parameters, not detection thresholds. Their generalization to other values is untested.

V. Proposed Methodology

A. Challenge Construction and Observable Features

At the beginning of a dense audit cycle, the server freezes its current global state $w^{(c)}$ and produces two independent perturbed states. For floating tensor k and trap $a \in \{A, B\}$,

$$\tau_{a,k} = w_k^{(c)} + \gamma \max\{\text{sd}(w_k^{(c)}), \epsilon_\tau\}\xi_{a,k}, \quad \xi_{a,k} \sim \mathcal{N}(0, I), \tag{6}$$

with $\gamma = 0.01$ and $\epsilon_\tau = 10^{-4}$. These are fixed scale choices, not an optimized trap-strength claim. The traps are independent draws, not opposite perturbations. All floating state tensors are included, including BatchNorm buffers in CIFAR-10; integer counters remain unchanged.

For a response, tensor-profile coordinate k is

$$p_{i,k} = \frac{\|\Delta_{i,k}\|_2}{\sum_j \|\Delta_{i,j}\|_2}. \tag{7}$$

A denominator below 10^{-6} produces a zero profile. This numerical convention is distinct from exact-sendback evidence. Cosine similarity is neither collected nor used by this version.

Only responses to the same trap are compared. For each client, all other available active clients form its leave-one-out reference population, without trusted-anchor or label filtering. Let m_i and d_i be the peer norm median and median absolute deviation (MAD). The robust norm score is

$$z_i^n = \frac{r_i - m_i}{d_i / \Phi^{-1}(0.75)}. \tag{8}$$

The normal-consistency factor rescales MAD; it does not assume that updates are Gaussian. Zero MAD yields an undefined score rather than a fitted floor. Profile distance is measured from the peer coordinate-wise median profile; its score z_i^p uses the median/MAD of peer distances about that same reference.

B. Phase I: Warm-Up

Rounds 1–10 use ordinary global models. Exact send-backs may be recorded, but warm-up observations cannot bypass later confirmation or authorize removal. This phase supplies an initial training trajectory and received-model history. It does not estimate a trusted honest population or use private client data to calibrate a detector.

C. Phase II: Initial Two-Trap Coverage

During rounds 11–20, ten shuffled groups provide every client one response to each trap. Five paired-group assignments are followed by assignments using the opposite trap, with randomized order/signs. Approximately 20% of clients are trapped each round in the 100-client initial population. Their responses are withheld from aggregation. Decisions wait for complete A/B coverage.

Initial nomination uses $D_i = z_i^p - z_i^n$ from trap A, trap B, and their average. Each view uses $\lceil\sqrt{N}\rceil$ equal-width histogram bins and an Otsu between-class-variance split [10]. Whole-client bootstrap resampling [11] uses 1,000 replicates and the same resampled client indices across the three views. A point above the upper 99% boundary-stability band in every view is nominated as suspicious, one below every lower band is unflagged, and disagreement is candidate evidence. A valid split also requires the low-score side to contain a majority of the population. If the split is unsupported, below-peer-median norm on both traps nominates suspicion, one/undefined response gives C, and neither gives R. Exact current/previous sendback on both traps is handled separately.

The bootstrap describes stability of the estimated boundary, not a 1% false-positive probability. Neither a histogram split nor its fallback can directly authorize statistical removal.

D. Geometric Separation Certificates

Let $\mathcal{C}$ be the nominated suspicious cohort and $\mathcal{P}$ its complement on one trap. Require

$$\left\lceil \frac{N}{\lceil\sqrt{N}\rceil} \right\rceil \le |\mathcal{C}| < |\mathcal{P}|. \tag{9}$$

The lower bound is a population-derived support choice, not knowledge of the number of attackers. It can prevent certification of isolated free riders. For cohort IQR $I_C = Q_{.75}(\mathcal{C}) - Q_{.25}(\mathcal{C})$, low raw update norms certify only when

$$Q_{.25}(\mathcal{P}) - Q_{.75}(\mathcal{C}) > I_C, \tag{10}$$

$$\min(\mathcal{P}) - Q_{.75}(\mathcal{C}) > I_C. \tag{11}$$

The individual must also lie below the peer median. This strict full-peer gate rejects norm evidence based solely on a compact but overlapping cohort. It separates a robust core, not necessarily every cohort member, because its upper quartile is not its maximum.

For high raw profile distance, write I_P for peer IQR and $g = Q_{.25}(\mathcal{C}) - Q_{.75}(\mathcal{P})$. Certification requires $g > I_C$, $g > 0$, and at least one of

$$Q_{.25}(\mathcal{C}) - \max(\mathcal{P}) > I_C, \quad \text{or} \tag{12}$$

$$\frac{I_C}{|\mathcal{C}| - 1} < \frac{I_P}{|\mathcal{P}| - 1}. \tag{13}$$

An individual must exceed the peer median. The concentration alternative remains in the profile gate; it is not retained in the norm gate. The same component must certify on both traps. Diagonal scores remain diagnostic rather than an independent removal component, because they already contain norm evidence. The term certificate denotes this operational geometric test, not proof of dishonest behavior.

E. Phase III: Confirmation, Memory, and Bounded Review

For each client, retain separate counts $M_{i,n}$ and $M_{i,p}$ for full A/B norm and profile certificates. A fresh certificate increments only its own component once per distinct completed cycle. C/ambiguous cycles retain counts without incrementing them; two consecutive full R cycles clear statistical memory. Removal requires a fresh current certificate whose component count reaches two. Thus, norm evidence followed by profile evidence does not combine into confirmation. Exact zero delta or fingerprint equality with the immediately previous sent state uses a separate consecutive-full-cycle confirmation path. No one-trap audit can confirm removal. The earliest possible removal is round 30.

A fresh strong certificate starts a two-cycle quarantine/review budget. Ambiguous evidence and stored counts cannot renew that budget; if fresh strong evidence does not return, exclusion expires. Weak candidates are allowed to aggregate on ordinary rounds. Complete clean cycles rehabilitate a client, with one additional clean probation cycle before the R-state reference-eligibility marker is restored. That marker does not filter the current all-active peer reference. An audit-based cooldown of two single-trap audits limits immediate weak re-enrolment; fresh paired strong evidence can still trigger review.

Later dense cycles keep monitored clients and ordinary clients in separate groups, with at most ten nonempty groups allocated proportionally to pool size. Monitored clients receive alternating A/B traps in all ten rounds; ordinary clients receive each trap once. One response position per client per trap is selected before scores are observed, so extra monitoring responses cannot become extra confirmation strikes. Fresh traps and shuffled groups are used each cycle. Algorithm 1 summarizes the removal logic.

F. Phase IV: Sparse Audits and Aggregation

When neither bounded review nor clean-cycle probation remains, the server uses a single-trap all-active audit every ten rounds. This is one audit round, not one group per round. Consequently, no ordinary update aggregates in that

Algorithm 1 Full-cycle evidence and permanent-removal decision

Require: complete selected A/B responses, distinct new cycle c

derive same-trap norm/profile statistics and nominate clients
test cohort separation independently for norm and profile
for all active clients i do
    for all components $j \in \{n, p\}$ freshly certified on A and B do
        $M_{i,j} \leftarrow M_{i,j} + 1$
        if $M_{i,j} \geq 2$ then
            permanently remove i
        end if
    end for
    if complete R decision then
        advance consecutive clean-cycle streak
        if two clean cycles then
            clear $M_{i,n}, M_{i,p}$
        end if
    else
        interrupt clean streak
    end if
    apply separate consecutive-cycle exact-sendback rule
    start/renew review only with fresh strong evidence
    expire unconfirmed exclusion when review budget ends
end for
aggregate eligible ordinary responses only

audit round. A single-trap anomaly can initiate bounded follow-up but cannot remove a client or clear statistical memory. If follow-up is required, dense cycles resume; otherwise sparse auditing continues.

Let $\mathcal{A}_t$ contain active clients that were neither trapped nor quarantined, excluding new decision-round removals/quarantine. Ordinary updates are equally averaged:

$$w_{t+1} = w_t + \frac{1}{|\mathcal{A}_t|} \sum_{i \in \mathcal{A}_t} \Delta_{t,i}. \tag{14}$$

If $\mathcal{A}_t$ is empty, the global model is unchanged. Equal weighting is explicit: the server does not use private sample counts. Trap training replaces that client's ordinary training in a round; it is not a second local training job.

VI. Results

A. Experimental Setup and Reporting Scope

The completed grid is 2 datasets $\times 2$ partitions $\times 2$ free-rider shares $\times 4$ attacks, giving 32 runs. Every run has 100 clients and 100 rounds. MNIST uses handwritten digit images; CIFAR-10 uses ten-class color images [12], [13]. IID data are shuffled into equal-size client partitions; non-IID data use class-wise Dirichlet allocation with $\alpha = 0.5$, retrying for nonempty clients.

All runs use master seed 42, with separate derived streams for partitioning, attacker selection, model initialization, training, grouping, trap generation, and attack randomness. The policy was developed after inspecting earlier seed-42 failures, so these are development evaluations, not held-out-seed tests. All scheduled runs are included. No best-seed selection or replacement with an older method is performed.

Honest clients use three local SGD epochs, learning rate 0.01, momentum 0.9, and batch size 32. MNIST uses two convolution blocks (16 and 32 channels) and a 128-unit hidden classifier. CIFAR-10 uses three convolution/BatchNorm blocks (32, 64, 128 channels) and a 256-unit hidden classifier. Images use fixed normalization without augmentation. The reported executions all resolve to CPU.

B. Evaluation Metrics

A positive detection is permanent removal, not nomination or quarantine. For true positives TP, honest removals FP, and missed free riders FN,

$$P = \frac{TP}{TP + FP}, \qquad R = \frac{TP}{TP + FN}, \qquad F_1 = \frac{2TP}{2TP + FP + FN}. \tag{15}$$

Precision is undefined if no client is removed. Complete-detection round t_{all} exists only when every free rider has been removed; otherwise it is NR. Final global test accuracy is a separate utility metric. We additionally report final honest-candidate counts, honest clients ever quarantined, quarantine client-rounds, trap client-rounds, and all excluded active honest client-rounds. These last categories can overlap and must not be summed as disjoint costs.

C. Permanent Detection and Model Utility

Table I reports all eight dataset/partition/share groups, each containing four attack runs, with a separate final-accuracy column for FR1–FR4 and a mean round-time column. Across the full grid, 1120 free-rider client-cases are removed, with 0 honest removals and 0 misses. Pooled precision, recall, and F_1 are therefore 100% for this completed grid. The 2080 honest client-cases are repeated experimental observations, not an independent population of that many people or devices.

All sixteen MNIST executions remove every free rider at round 30. CIFAR-10 also finishes at round 30 except for non-IID FR3: round 60 at 40% and round 80 at 30%. The largely coincident cumulative removal curves are retained in Appendix B, Figure 10, rather than using a main-text figure to repeat perfect final detection. Thus, a perfect final recall conceals a substantial difference in latency.

Final accuracy ranges from 98.77% to 99.02% on MNIST and from 74.89% to 78.17% on CIFAR-10. Figure 5 presents the corresponding late-training trajectories on panel-specific percentage-point scales; full curves remain in the appendix. No honest-only or matched no-detector execution is included in this grid, so these accuracies do not establish a causal improvement attributable to the defense.

D. Observed Score Geometry and Evidence Evolution

Figures 1 and 2 show every client's round-20 norm/profile position at 40%, covering all four attacks and both partitions. The 30% counterparts are retained in Appendix B. Coordinates are paired averages over the two selected trap responses, transformed by asinh solely for display. This monotone transformation preserves ordering while keeping extreme profile-z outliers from flattening the central cloud. There is no clipping, jitter, label-based reference selection, or fitted decision line. Coincident points, particularly exact sendbacks, can conceal multiplicity. The actual decision continues to test each trap separately; a paired scatter point is a diagnostic, not a substitute for the full certificate.

The more demanding CIFAR-10 non-IID FR3 executions are examined directly in Figure 3. Shared population bins expose raw trap-A norm distributions before and at confirming cycles, while annotations report the paired fresh norm-certificate counts from the decision logs. The certificate can include honest clients: a visually compact low-norm population is not equivalent to a pure attacker cluster. At 40%, nine honest clients receive the round-20 paired norm certificate, but none receives it at round 60. This explains why nomination quality and eventual removal quality differ. At 40%, the round-20 and round-60 certificates confirm all free riders; at 30%, the relevant pair is round 40 and round 80. The selected panels explain this recorded mechanism, not a new decision rule selected from the histograms.

Figure 4 supplements the raw distributions with class mean norm-z and profile-z over complete cycles, with interquartile client dispersion. Class labels are used only after scoring to describe observed behavior; the server never calculates an "honest-only" boundary. A lower free-rider mean alone is insufficient: the strict certificate must still establish cohort support and separation from the peer range on both traps. Because reference populations and MAD scales change after removal, z-score trajectories are not fixed-unit measurements of model improvement. Undefined scores remain missing. The first ten warmup rounds do not provide these paired trap scores, and no warmup score trajectories are fabricated.

E. Honest-Client Impact

Zero permanent honest removals does not mean zero false suspicion. Table II expands FR3 into individual dataset/partition/share rows rather than pooling it with the three easier attacks. It distinguishes distinct quarantined clients, quarantine duration, deliberate trap exclusion, and additional non-trap exclusion. Across the full grid there are 11 honest client-cases ever quarantined, totalling 220 quarantine client-rounds. Nine occur in CIFAR-10 non-IID FR3 at 40%, one in its 30% counterpart, and one in MNIST non-IID FR3 at 40%. None remains quarantined at completion.

There are 296 final honest-candidate client-cases across the 32 runs. Weak candidate labels persist in several completed runs. The most pronounced cases are 34 of 60 honest clients in MNIST non-IID FR3 at 40%, 52 of 60 in CIFAR-10 non-IID FR3 at 40%, and all 70 remaining clients in CIFAR-10 non-IID FR3 at 30%. These are not counted as permanent false positives, but they are reported because screening and final-removal quality are different properties.

F. Execution Cost

Table III reports means across the four attacks, giving each attack equal weight, with the attack-to-attack range of both round time and peak RSS. MNIST requires approximately 34.6–36.4 seconds per round at 40% free riders and 41.3–43.0 seconds at 30%. CIFAR-10 requires 79.5–90.7 and 97.1–101.2 seconds, respectively. The measurements include serial client simulation, local training, aggregation, and evaluation. The logged round timer ends before checkpoint/log publication; reported execution duration additionally includes initialization, publication work, and recovery work represented by the resumed timer. Powered-off waiting is not a detector computation cost.

Process RSS includes simulated attacker histories and training tensors, not only server-side detection state. Neither round duration nor peak RSS isolates the defense's incremental overhead. A detector-only profiler and controlled no-detector baseline are required before assigning an overhead percentage. Figure 6 keeps the per-attack breakdown behind the grouped table: the higher-memory FR3/FR4 executions include simulated attacker histories as well as training and detector tensors. It would be incorrect to attribute that entire difference to server-side detection.

Table IV compares the three completed same-setting v15.3 CIFAR-10 non-IID 40% executions against v15.4. The unfinished FR1 is excluded explicitly, not treated as a completed measurement. This historical comparison is useful context for the concern that evidence memory might be expensive; it is not an isolated overhead experiment. Different detection trajectories, client work, restarts and host load prevent interpreting the differences as a causal speedup. Neither historical nor current logs measure network bytes or detector-only CPU time. Section VIII therefore separates measured process resources from calculated algorithmic requirements.

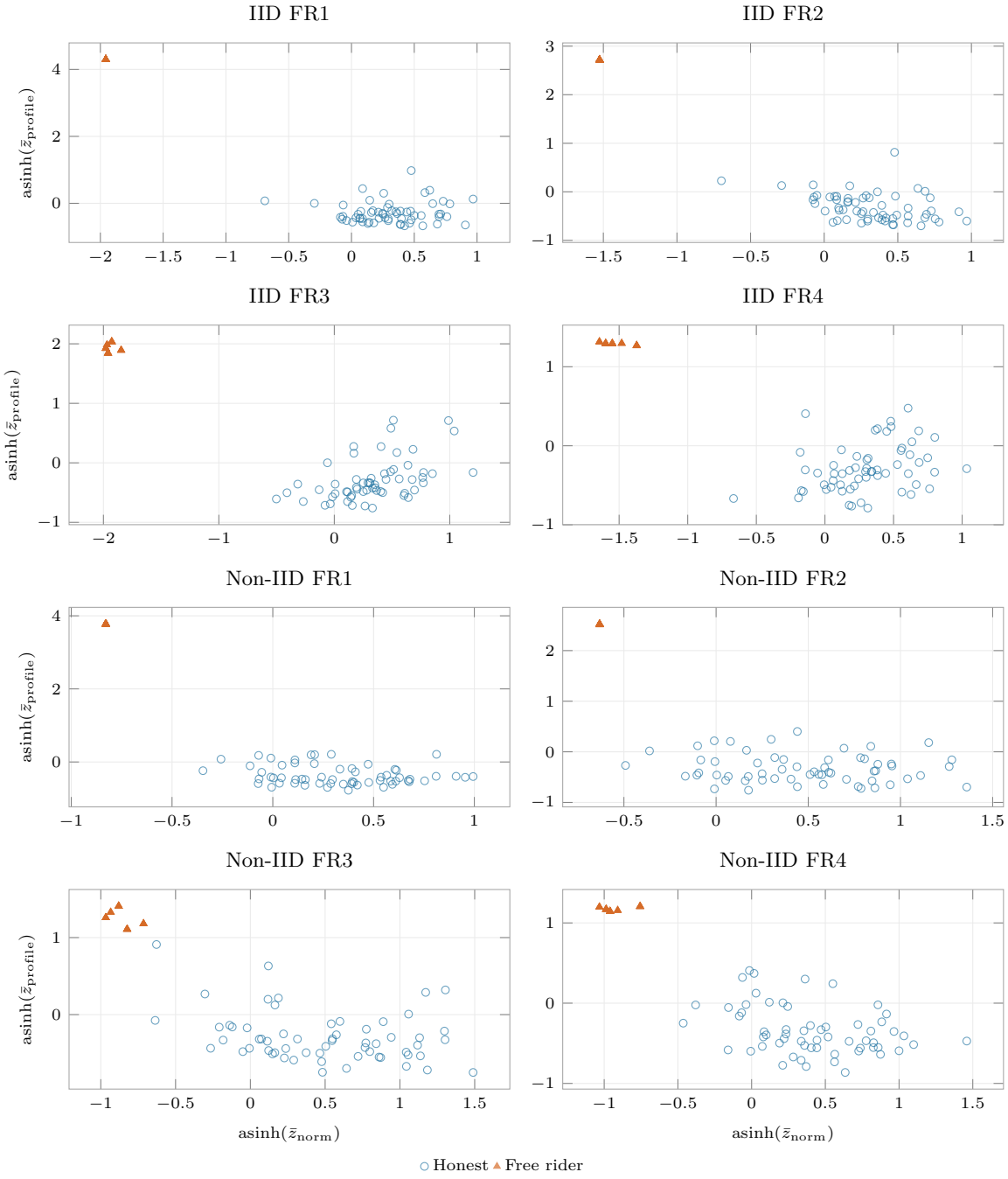


Fig. 1. MNIST: round-20 score geometry at 40% free riders (all eight settings). One point is one client, averaging the A/B norm-z and profile-z scores separately. The monotone asinh display compresses extreme tails without clipping; zero remains zero, but distances are transformed. Panels autoscale independently. Labels color evaluation only, never references or decisions. 0 points have undefined coordinates and are omitted; coincident points overlap. No plotted line is a removal cutoff.

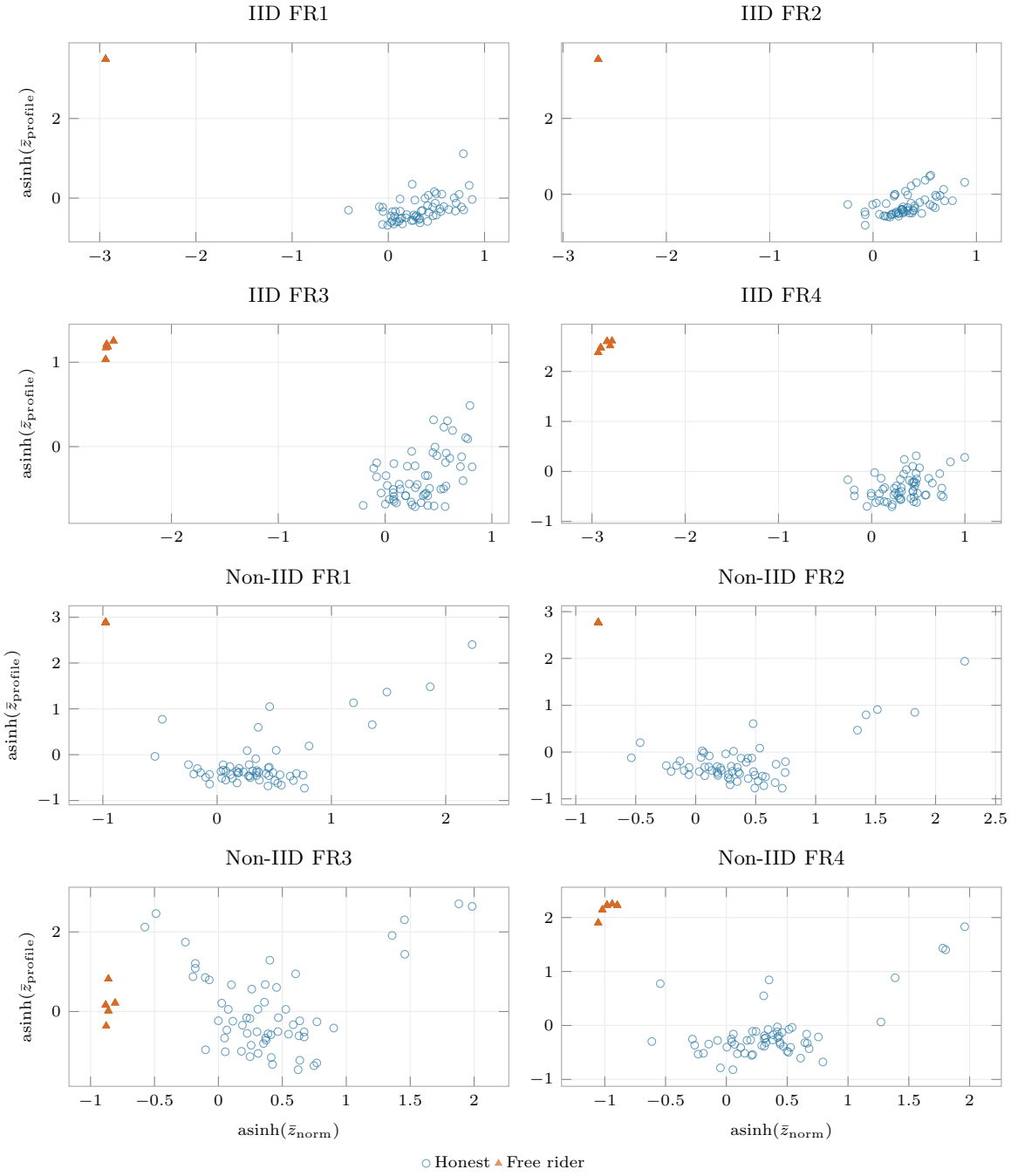


Fig. 2. CIFAR-10: round-20 score geometry at 40% free riders (all eight settings). One point is one client, averaging the A/B norm-z and profile-z scores separately. The monotone asinh display compresses extreme tails without clipping; zero remains zero, but distances are transformed. Panels autoscale independently. Labels color evaluation only, never references or decisions. 0 points have undefined coordinates and are omitted; coincident points overlap. No plotted line is a removal cutoff.

TABLE I
Detection quality, attack-specific final test accuracy (%), and mean round time. Each row groups four attacks; seed 42, 100 rounds.

Dataset	Partition	FR (%)	Precision (%)	Recall (%)	FR1 acc.	FR2 acc.	FR3 acc.	FR4 acc.	s/round
MNIST	IID	30	100.00	100.00	99.02	99.00	99.00	98.82	42.99
MNIST	Non-IID	30	100.00	100.00	99.01	99.00	98.98	98.96	41.28
MNIST	IID	40	100.00	100.00	98.90	98.91	98.85	98.77	36.43
MNIST	Non-IID	40	100.00	100.00	98.96	98.96	98.93	98.98	34.57
CIFAR-10	IID	30	100.00	100.00	78.07	77.93	78.17	78.03	101.23
CIFAR-10	Non-IID	30	100.00	100.00	75.80	75.58	75.36	75.78	97.15
CIFAR-10	IID	40	100.00	100.00	77.46	77.65	77.27	77.36	90.66
CIFAR-10	Non-IID	40	100.00	100.00	75.28	75.43	74.89	75.14	79.46

Precision/recall pool permanent removals over FR1–FR4; accuracy measures the global model, not the detector. Time averages the four logged mean round times. Detailed counts and completion rounds are retained in Appendix A.

TABLE II
Honest-client burden by individual FR3 run, plus any other run with honest quarantine. All runs end with zero permanent honest removals and zero quarantined clients.

Dataset	Partition	FR (%)	Attack	Final honest candidates	Ever quarantined	Quarantine client-rounds	Trap client-rounds	Non-trap excluded	All excluded (%)
MNIST	IID	30	FR3	8/70	0	0	770	0	11.00
MNIST	Non-IID	30	FR3	8/70	0	0	770	0	11.00
MNIST	IID	40	FR3	5/60	0	0	660	0	11.00
MNIST	Non-IID	40	FR3	34/60	1	20	736	1	12.28
CIFAR-10	IID	30	FR3	2/70	0	0	770	0	11.00
CIFAR-10	Non-IID	30	FR3	70/70	1	20	1160	0	16.57
CIFAR-10	IID	40	FR3	4/60	0	0	660	0	11.00
CIFAR-10	Non-IID	40	FR3	52/60	9	180	1104	5	18.48

Candidate entries are unresolved honest clients / honest population. Quarantine client-rounds count clients times rounds, and can overlap trap client-rounds. Non-trap excluded counts only withheld honest responses not already explained by traps; the last column divides all withheld honest responses by active honest client-rounds. The complete per-run appendix retains the other attacks.

TABLE III
Measured whole-simulator resource comparison on CPU: mean [minimum–maximum] across FR1–FR4; execution is mean minutes per run.

Dataset	Partition	FR (%)	s/round: mean [range]	Peak RSS GiB: mean [range]	min/run
MNIST	IID	30	43.0 [42.3–44.0]	0.53 [0.29–0.72]	75.9
MNIST	Non-IID	30	41.3 [40.7–41.7]	0.56 [0.30–0.75]	73.2
MNIST	IID	40	36.4 [35.3–39.6]	0.59 [0.34–0.85]	65.6
MNIST	Non-IID	40	34.6 [34.3–34.9]	0.61 [0.31–0.85]	62.4
CIFAR-10	IID	30	101.2 [88.2–119.1]	1.28 [0.76–1.75]	178.6
CIFAR-10	Non-IID	30	97.1 [89.6–108.4]	1.30 [0.74–1.76]	170.4
CIFAR-10	IID	40	90.7 [77.4–104.2]	1.38 [0.72–1.93]	165.7
CIFAR-10	Non-IID	40	79.5 [75.8–84.8]	1.50 [0.78–2.13]	143.6

Peak RSS includes local training, aggregation, logged evidence and simulated attacker histories. Round timing ends before checkpoint/log publication. Neither column isolates detector overhead or controls machine load.

G. Measured 100-Round Native-Workflow Timing

We additionally execute 100 ordinary-update STD-DAGMM rounds per dataset: 100 clients throughout, non-IID, seed 42, 40% FR3, three local SGD epochs, CPU and six PyTorch threads. This is a paper-guided implementation, not author code. Each round freshly fits the full-update autoencoder/mixture for a declared budget of five epochs and records fitting and scoring separately. This budget is not evidence of detector convergence or an exact replication of an unspecified published training schedule. Flags remain observational: no traps, our evidence gates, or permanent removal are added.

Table V and Figure 7 distinguish measured components from missing measurements. Our completed study logged whole-round durations, not separate client-training, aggregation, trap-generation or detection timers; gray totals cannot retrospectively identify those costs. Our feature collection occurs during client responses, whereas complete-cycle scoring and state decisions occur at coverage boundaries; single audits are separate events. Therefore detection is not simply zero on every non-tenth round. Neither local simulator measures real network transmission. STD input serialization and worker startup are harness work, not network communication.

For our actual horizon, the amortized time is $\bar{t}_{100} = \sum_{r=1}^{100} t_r/100$; summing all rounds includes expensive probing/decision rounds and lighter rounds without selecting either subset. Table VII and Figure 8 explain how round

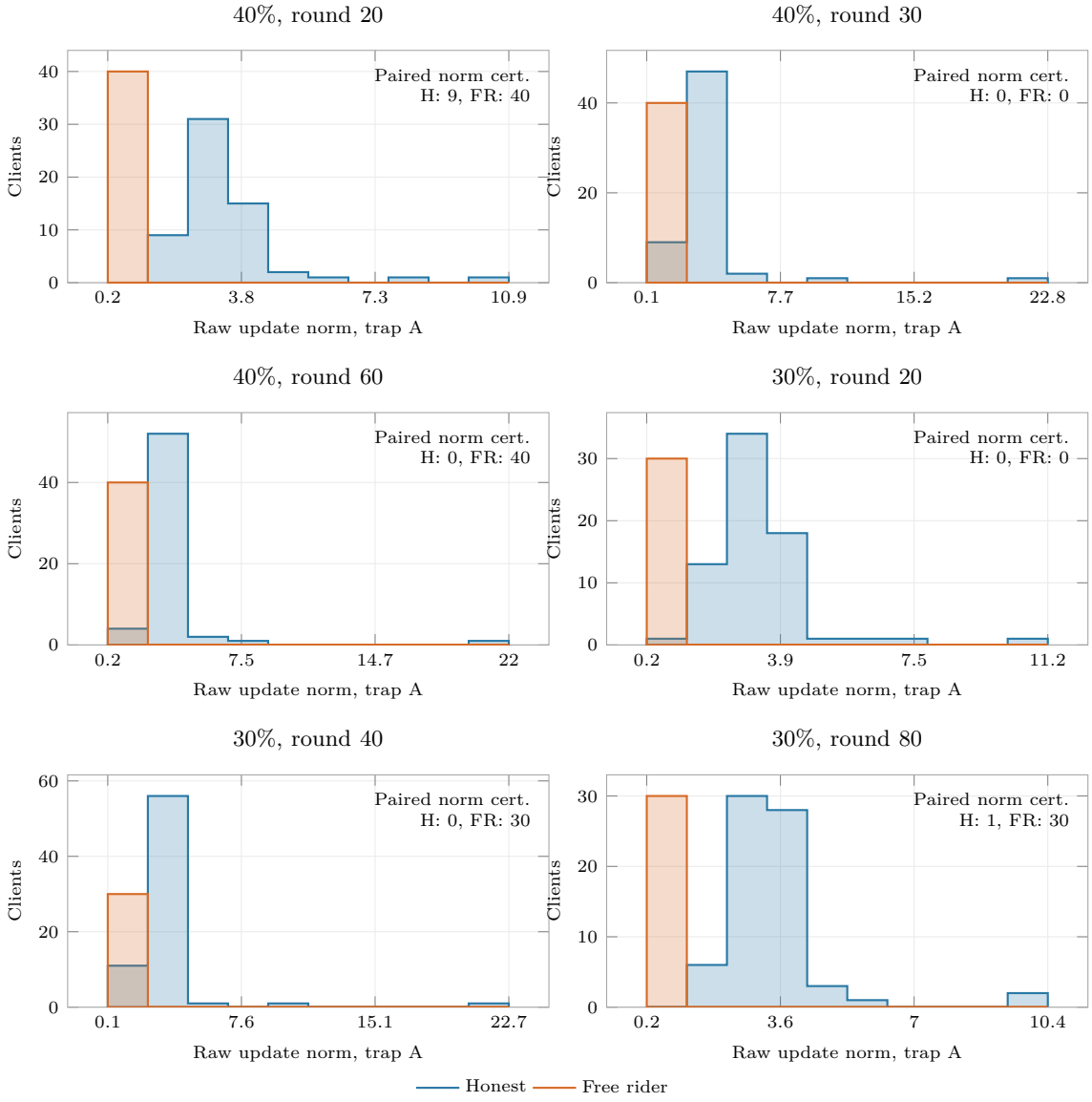


Fig. 3. CIFAR-10 non-IID FR3: raw selected trap-A norm distributions. Shared population-derived equal-width bins within each panel, independent axes across panels; colors may overlap. Annotations count FRESH norm certificates on BOTH traps (H: honest, FR: free rider). Confirming cycles are 20/60 at 40% and 40/80 at 30%; the other panels show ambiguity. A visual gap alone cannot authorize removal.

frequency affects this average. The figure's left panels show the mean duration of a whole round in each category; its right panels multiply that mean by the category's round count and divide by 100. The five right-hand bars therefore sum to the observed full-run mean. For example, MNIST's three cycle-decision rounds average 34.45 seconds each and contribute 1.03 seconds to the 34.41-second full-run average. The 1.03 seconds is not an estimate of detector-only cost. All workflow comparisons use the full 100 executed rounds per method and dataset. Figure 9 shows both complete trajectories.

These are native-workflow costs, not a controlled speedup experiment: our trajectory removes clients and changes probing/aggregation, while the STD execution retains all 100. Historical local attacker evaluation, timer coverage, machine load and data trajectories also differ. CIFAR's detector has a sampled peak worker RSS of about 3.66 GiB and minimum available system memory near zero; paging may confound timing, although paging itself was not measured. This experiment does not establish universal STD-DAGMM detection quality or end-to-end removal effectiveness. Per-round components, phase sums and input hashes are retained with the reporting artifacts. The measured mean round compute is 50.19 seconds on MNIST and 127.24 seconds on CIFAR-10; detector fitting contributes 13.63 and 45.03 seconds, respectively. In these selected non-IID FR3 runs, no free rider is ever flagged, whereas all 60 honest clients are flagged at least once

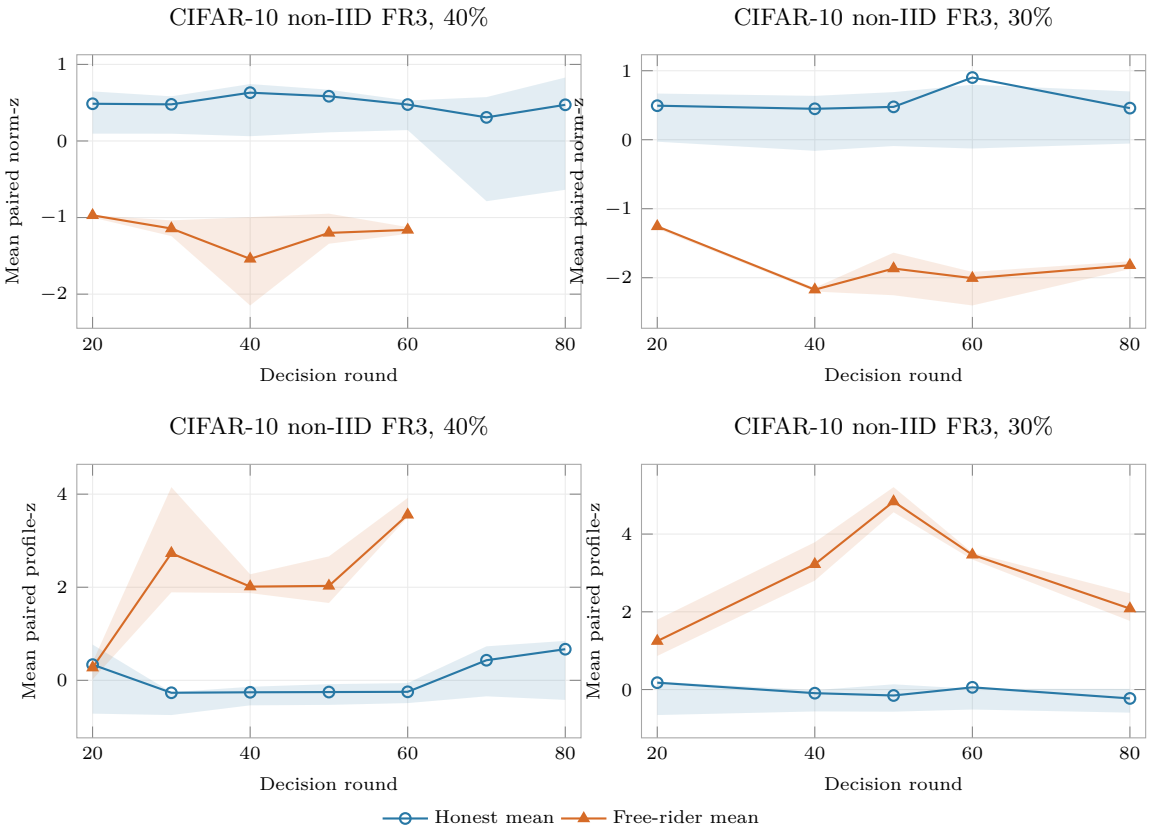


Fig. 4. CIFAR-10 non-IID FR3: mean paired scores over complete evidence cycles through round 80 (lines), with client interquartile intervals shaded. Scores are first averaged over A/B per client. Bands show client dispersion, NOT mean uncertainty. References/MAD scales change each cycle; z-score changes need not match raw-norm changes. Free-rider curves stop at removal, never zero-filled. Class colors are for evaluation only.

TABLE IV
Historical whole-process comparison: CIFAR-10, non-IID, 40%, seed 42, CPU, 100 rounds. Only completed matching v15.3 runs are included.

	Mean s/round		Peak RSS (GiB)		Execution (min)	
Attack	v15.3	v15.4	v15.3	v15.4	v15.3	v15.4
FR2	78.32	75.80	0.78	0.78	133.06	128.91
FR3	85.32	84.85	1.89	2.13	163.38	160.10
FR4	86.50	78.79	1.81	2.07	169.51	149.52

The unfinished v15.3 FR1 is not reconstructed. Different trajectories, machine load and restarts confound causal overhead claims. These are not isolated detector benchmarks.

(not all in the same round). These observational flags are not removals and do not validate the chosen five-epoch recipe as an effective detector; neither do they establish a universal limitation of STD-DAGMM. Removing non-training free riders does not itself remove honest SGD jobs: the savings can concern response handling, simulated attacker diagnostics, aggregation and detection, rather than a proportional reduction of genuine local-training work. The observations cannot isolate those causes.

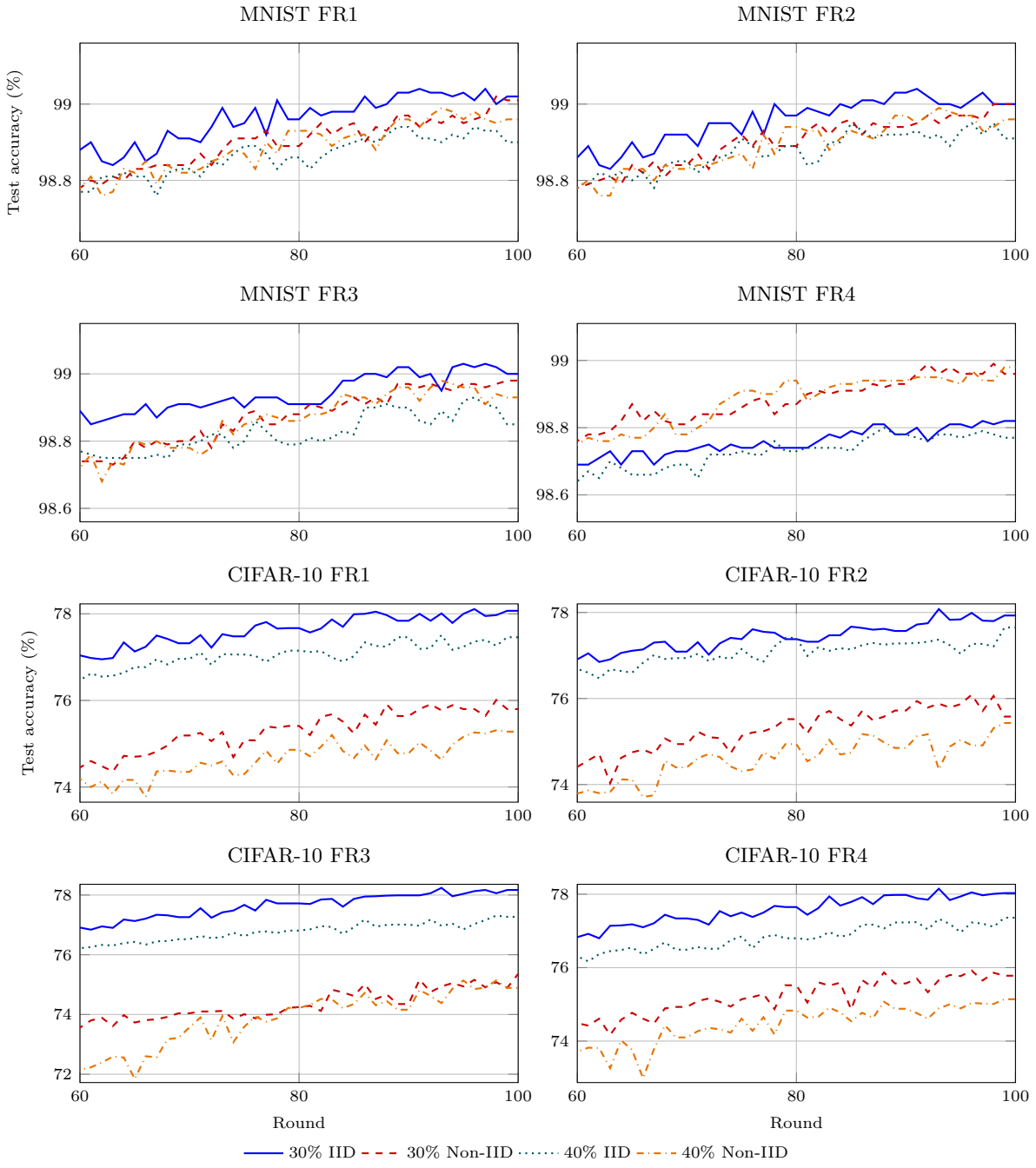


Fig. 5. Late-training global test accuracy (rounds 60–100; all 32 runs). Each panel uses its OWN data-derived vertical range in percentage points to expose differences hidden by a 0–100 percent scale. Compare numeric ticks, not apparent slope across panels. Full trajectories appear in the appendix; no smoothing or runs are omitted.

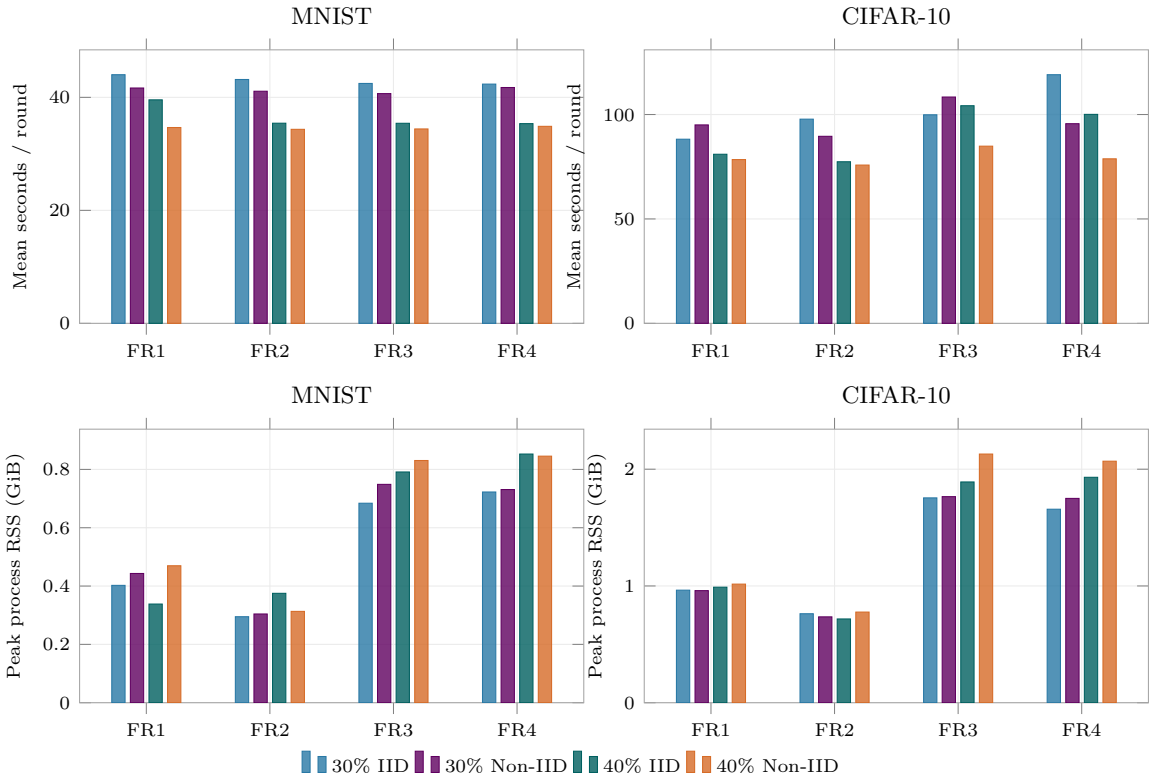


Fig. 6. Attack-specific whole-simulator resource profiles for all 32 runs: mean logged round time (top) and peak process RSS (bottom). Four bars per attack distinguish partition and free-rider share. Dataset panels use independent numeric scales. FR3/FR4 memory includes simulated history storage; these bars do not measure detector-only overhead.

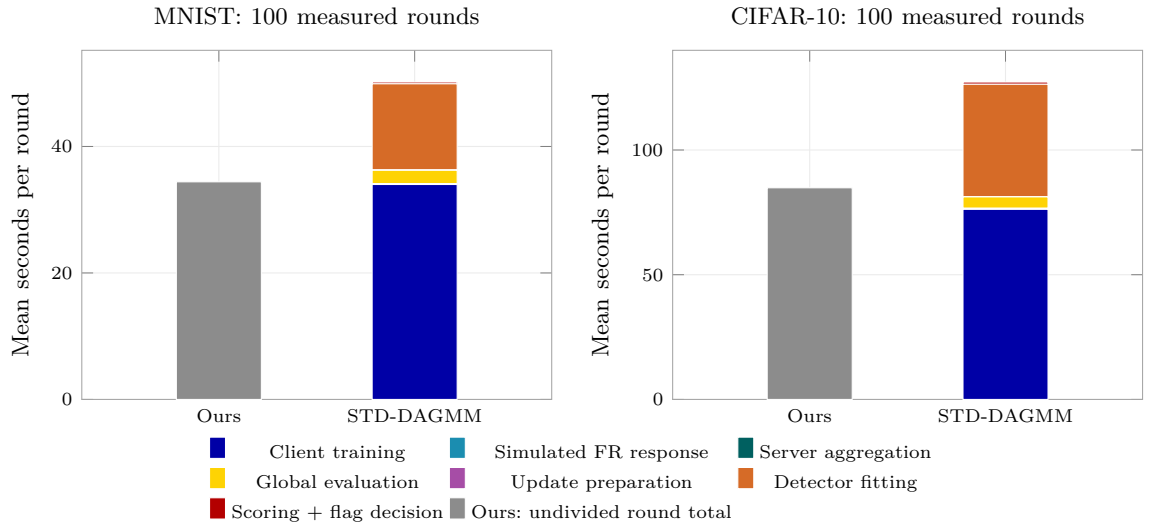


Fig. 7. Mean timing over 100 executed rounds per method and dataset: non-IID FR3, seed 42, 100 initial clients, 40% free riders. STD-DAGMM has measured task components; our gray bars are undivided whole-round totals, not zero-cost detection. Both means include rounds 1–100. Different trajectories, timer coverage and host load prevent a causal speedup claim.

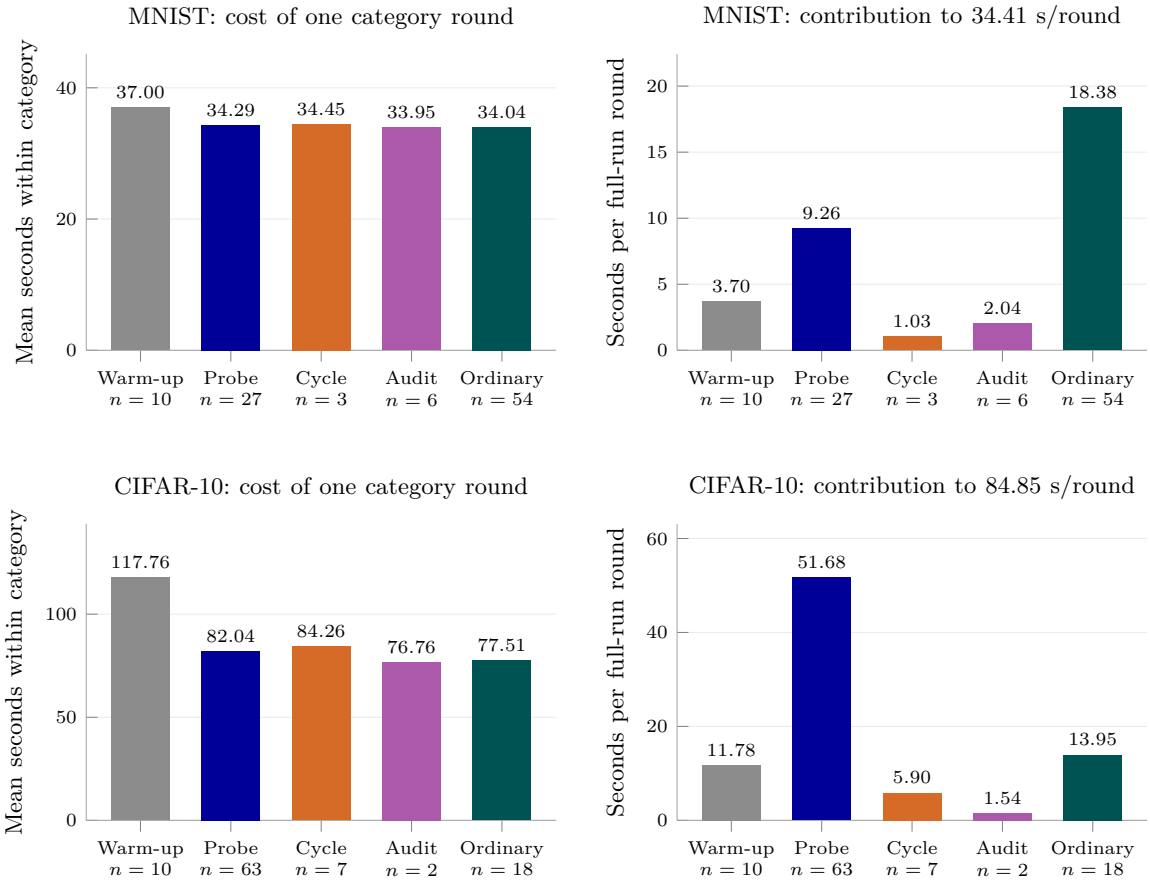


Fig. 8. Where our 100-round runtime is spent (non-IID FR3, seed 42, 40% free riders). Rows separate MNIST and CIFAR-10. Left: mean duration of a complete round in each category. Right: summed duration for that category divided by 100; the five bars add to the full-run mean printed in the title. Labels give category counts n. Probe excludes cycle-decision rounds; Cycle includes the final cycle decision; Audit is a single audit round. Categories are mutually exclusive. A cycle bar includes client training, aggregation and evaluation: it is NOT detector-only time. Axes scale independently; all bars start at zero.

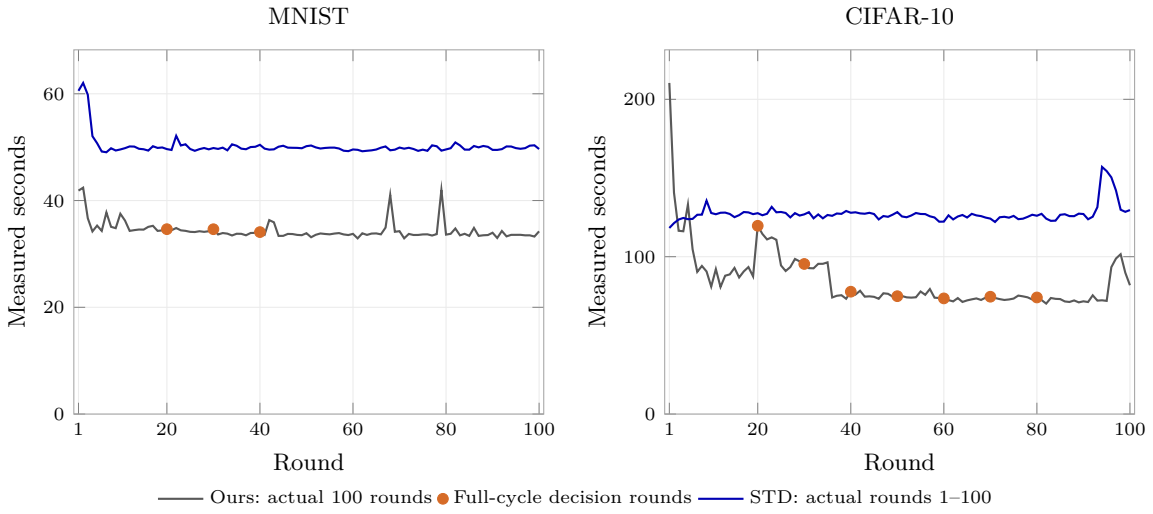


Fig. 9. Actual per-round durations over 100 executed rounds for both methods. Our trajectories include changing active-client counts and probing schedules; STD-DAGMM retains all 100 clients. Markers indicate full-cycle decisions, not their isolated computational cost. Axis scales differ by dataset.

TABLE V
Measured STD-DAGMM component times (seconds per round, mean ± sample SD, 100 rounds per dataset).

Component	MNIST	CIFAR-10
Client training	33.986 ± 1.539	76.208 ± 4.164
Simulated FR response	0.062 ± 0.006	0.255 ± 0.055
Server aggregation	0.042 ± 0.004	0.189 ± 0.043
Global evaluation	2.144 ± 0.073	4.547 ± 0.227
Update preparation	0.045 ± 0.004	0.127 ± 0.015
Detector fitting	13.629 ± 0.467	45.029 ± 1.671
Detector scoring	0.278 ± 0.011	0.856 ± 0.035
Flag decision	0.000 ± 0.000	0.027 ± 0.024
All detector work	13.952 ± 0.476	46.040 ± 1.695
Combined compute	50.186 ± 1.943	127.239 ± 5.478

Our historical training, aggregation, detection and trap-generation components: not separately recorded. Network transmission: not measured in either local simulator. STD has no traps. All detector work includes preparation, fitting, scoring and decisions; it overlaps those table rows and must not be added a second time.

TABLE VI
Native-workflow timing over 100 executed rounds per method and dataset. All entries are measured; this is not a controlled speedup experiment.

Dataset	Workflow / horizon	Rounds	Mean s/round	Sum (min)
MNIST	Ours, measured full horizon	100	34.41	57.35
MNIST	STD-DAGMM, measured full horizon	100	50.19	83.64
CIFAR-10	Ours, measured full horizon	100	84.85	141.41
CIFAR-10	STD-DAGMM, measured full horizon	100	127.24	212.06

TABLE VII
Our 100 measured rounds grouped by events. Within-category mean is not detector-only overhead; amortized contribution is category sum / 100.

Dataset	Round category	Count	Within-category mean (s)	Contribution (s/round)
MNIST	Warm-up	10	37.00	3.70
MNIST	Probe without cycle decision	27	34.29	9.26
MNIST	Full-cycle decision	3	34.45	1.03
MNIST	Single audit	6	33.95	2.04
MNIST	Ordinary	54	34.04	18.38
CIFAR-10	Warm-up	10	117.76	11.78
CIFAR-10	Probe without cycle decision	63	82.04	51.68
CIFAR-10	Full-cycle decision	7	84.26	5.90
CIFAR-10	Single audit	2	76.76	1.54
CIFAR-10	Ordinary	18	77.51	13.95

VII. Result Analysis

A. Effect of Attack Behavior

FR1 is consistently removed at the earliest allowed round. Exact sendback provides a distinct evidence path, so its success does not validate the statistical separation gate. Near-converged honest zero updates would still require separate examination in other training regimes.

FR2 is removed by round 30 in all settings despite avoiding equality. The observed responses are sufficiently distinct under the configured random update distribution and repeated traps. This is evidence about the tested amplitude, not proof that arbitrary fabricated noise will be detected.

FR3 is the critical heterogeneous-data case. Its current-inclusive history can yield responses that overlap honest update geometry. In non-IID CIFAR-10 at 40%, all 40 free riders receive full norm certificates at rounds 20 and 60; at 30%, all 30 receive them at rounds 40 and 80. Ambiguous intervening cycles do not increment these counts. Retention permits the later fresh certificate to confirm removal. This recorded sequence explains how the memory rule operates; it is not a matched ablation proving that memory alone caused the outcome under a counterfactual training trajectory.

FR4 is removed by round 30 in the present grid. Its received-trajectory estimate includes traps, which plausibly contributes to separation from genuine optimization. This is an interpretation of the mechanism, not a measurement of the attacker's inference ability. A defense-aware attacker could treat repeated or unusual received states differently and needs a separate evaluation.

B. Dataset, Partition, and Attacker Share

MNIST completes removal early in every condition, while non-IID CIFAR-10 FR3 requires additional evidence cycles.

This shows that neither attack identity alone nor final recall fully describes difficulty. The interaction between honest-data heterogeneity, model state, and fabricated response geometry matters.

Changing the share from 40% to 30% preserves complete detection but delays the difficult CIFAR-10 FR3 case from round 60 to round 80. A lower attacker share therefore does not imply faster discovery. Cohort support and the distribution of the surrounding honest population both change; the server does not use the known simulation share to force a quota or boundary.

The 30% runs also train 70 honest clients rather than 60. This increases genuine local work and helps explain longer simulator round times without implying a more expensive statistical rule. Non-IID client dataset sizes differ, so the increase need not equal the 16.7% change in honest-client count. The slightly higher achieved accuracies at 30% are consistent with more honest training, but no causal effect is established without matched controls.

C. Screening Quality Versus Removal Safety

The central positive result is that repeated-evidence gating prevents permanent honest removal in these 100-round trajectories, even when screening is noisy. The central unresolved issue is that some non-IID honest populations remain almost entirely candidate-labelled after attackers disappear. This can arise when relative statistical comparisons continue to find weak differences among honest clients. Such labels do not, by themselves, exclude ordinary updates, but they limit the meaning of calling R "safe" or C "malicious." We interpret R as unflagged, C as unresolved, and S as freshly certified suspicion.

Temporary exclusion also remains a real cost. Most honest exclusions are deliberate trap responses, not quarantine. In ordinary successful MNIST runs, 660 honest client-rounds are withheld at 40% and 770 at 30%, corresponding to eleven trap rounds per honest client over the 100-round horizon. The exceptional FR3 follow-up adds further probing. A deployment should measure whether this forgone aggregation delays reaching a target utility, rather than reporting permanent-removal precision alone.

D. What the Current Evidence Does Not Establish

The current grid has no held-out seed, honest-only control, long-horizon test, matched ablation, or baseline comparison under identical trajectories. We cannot estimate cross-seed variability, claim statistical independence of all client-cases, or infer universal 100% recall. Variations of attacker history length, noise amplitude, trap strength, local epoch count, non-IID concentration, and model architecture remain necessary robustness tests. In particular, the influence of CIFAR BatchNorm buffers should be separated from trainable-weight evidence.

VIII. Complexity and Lightweight Design

A. Operation Counts and Scaling

Let N be active clients, P floating model coordinates (including floating buffers), L floating tensors, T rounds, B bootstrap replicates and $K = \lceil\sqrt{N}\rceil$ histogram bins. The following are implementation bounds, not measured wall-clock predictions. We count model scanning separately from local optimization and ordinary aggregation.

The leave-one-out implementation rebuilds $N-1$ peer profiles for every client and trap. Profile medians and distances require $O(N^2L)$ coordinate work; repeated scalar medians in Python's statistics.median add $O(N^2 \log N)$ sorting work. Thus, calling the whole method linear in client count would be incorrect. Histogram binning and bootstrap resampling operate only on scalar scores, not on P-dimensional model vectors. With M complete cycles, the detector work is bounded by

$$\begin{aligned} W = {} & O\big(TNP\big) \\ & + O\big(M\left[N^2(L+\log N) + B(N+K)\right]\big)\,. \end{aligned} \tag{16}$$

Single-trap audits add corresponding one-view scoring passes; they cannot authorize permanent removal. Dense monitoring supplies more observations but only one preselected A/B pair per client enters each complete-cycle decision.

At $N = 100$, the two-trap profile comparison visits $2N(N-1)L = 158{,}400$ profile coordinates for MNIST and 435,600 for CIFAR-10, before constants for medians and distances. One bootstrap view draws $BN = 100{,}000$ scalar indices; three views process three such sets using reused draws in the current implementation. These counts explain why the evidence layer is compact relative to repeated full-model training, but they do not measure its elapsed time.

B. Payload Sizes, Memory Boundaries and Communication

Table IX derives concrete payload sizes from the implemented architectures. The compact float32 pair payload is $2N(L+3)\cdot 4$ bytes under the stated packed representation; the three scalar slots are an illustrative norm/round/flag allowance, not the actual Python object layout. The two trap models occupy $2P\cdot 4$ floating bytes. Actual dictionaries, lists, Python scalars, allocator overhead, fingerprints and bootstrap temporaries increase memory. For example, bootstrap index gathers and bin-index arrays coexist with resample/count arrays; their two listed sizes must not be relabelled as peak bootstrap RSS.

The decision working state is $O(P + NL + BN + BK)$, plus $O(N)$ counters and fingerprints. In contrast, the research logger accumulates per-round/probe rows, including tensor profiles, so the full simulator's retained evidence can grow as $O(TNL)$ in dense probing. Ordinary aggregation also retains model responses, and the simulator stores attack histories. The measured process RSS in Table III includes these costs; it is not evidence that the entire

TABLE VIII
Incremental detector operations and working state. "Cycle" means a complete A/B evidence cycle; it does not mean every training round. Bounds exclude ordinary model aggregation, local training, simulator attack histories and persistent logging.

Component	Work	Working state	Practical interpretation
Norm, profile, exact fingerprint	$O(NP)$ per round	$O(P)$ scratch, $O(N)$ fingerprints	Sequential model reads; no auxiliary training or gradients.
Selected A/B summaries	$O(NL)$ per cycle to store	$O(NL)$	Two compact profiles per active client, not full response histories.
Leave-one-out norm/profile scores	$O(N^2(L+\log N))$ per cycle	$O(NL)$ temporary	Repeated peer-profile medians/distances; Python scalar medians sort.
Three histogram/bootstrap views	$O(B(N+K))$ per cycle	$O(BN+BK)$ scratch	Three views add a constant factor; $B = 1000$.
Geometric gates and memory	$O(N)$ selections/quantiles per cycle	$O(N)$ counters	Fixed component count; no learned detector or history-window reconstruction.
Two model challenges	$O(P)$ generation per cycle	$O(P)$	Two shared trap states, not a distinct trap model per client.

TABLE IX
Calculated detector payload sizes at $N = 100$, two traps, $B = 1000$: architecture-derived estimates, NOT measured heap usage.

Model	Parameters	P	L	Two traps (MiB)	Pair payload (KiB)	Index array (MiB)	Counts (MiB)
MNIST	206922	206922	8	1.579	8.594	0.763	0.076
CIFAR-10	620810	621258	22	4.740	19.531	0.763	0.076

P includes floating BatchNorm buffers; L counts floating tensors. Pair payload assumes packed float32 profiles plus three scalar slots per trap/client. Bootstrap columns show two principal arrays, not peak scratch or an upper bound. No detector-only RSS was recorded.

implementation uses only compact decision-state memory. Streaming logs and aggregation would be deployment optimizations, not features benchmarked here.

No extra local optimization pass or auxiliary classifier is required. Under the existing full-participation request/response schedule, traps replace ordinary models rather than adding a second exchange; challenge responses are discarded from aggregation. The nominal model-payload order remains $O(NP)$ per round, excluding transport/serialization overhead. This does not make communication "free": useful contributions are withheld, so reaching a target accuracy can require more rounds. Network-byte measurements and controlled detector-only profiling are still necessary. We use lightweight to mean compact features, no auxiliary learning, and constant-sized per-client evidence memory, not an unmeasured universal speed or energy advantage.

IX. Security Analysis

A. Decision Inputs and Attack-Label Separation

The detector receives an allowlist of client/trap IDs, raw norms, profile distances, robust scores, and exact-sendback indicators. The simulator's attacker labels and behavior choices are not passed to screening, geometric tests, or the client state machine. Evaluation prevalence determines how many attackers the simulator creates, not how many removals the detector attempts to achieve. This code boundary reduces direct evaluation leakage; it does not make the development process independent of previously observed results.

B. Confirmation Invariants

Three operational safeguards follow from the state-transition rules. First, statistical removal requires the same component on both selected traps and a second distinct full cycle: one trap, mixed norm/profile evidence, or an ambiguous result cannot independently supply the required confirmation. Second, previously stored evidence does not authorize a new removal without a fresh current certificate. Third, stored counts and weak C evidence cannot renew quarantine; unconfirmed review expires unless fresh strong evidence returns. These are protocol invariants, not probabilistic security bounds.

The guarantees are intentionally narrower than "no false positives." An honest client can satisfy the same geometric gate in two cycles. A clean majority and independent trap draws do not make the resulting statistical certificates independent, because clients retain the same data and related model trajectories.

C. Evasion, Identifiability, and Trap Secrecy

Consider two executions with identical server-observable response histories, public identities, and server randomness, one produced by honest training and the other by fabrication. A detector whose transition function depends only on those observations has the same state trajectory in both executions. Therefore, update-only screening cannot prove genuine computation against every attacker. The proposed method must be evaluated by how well its challenges separate specified behaviors, not as a cryptographic proof of training.

The evaluated attackers are not given trap labels, but several real-world signals could reveal auditing: repeated frozen

states, abrupt distributional changes, or communication between clients receiving different models. Moreover, the simulator derives reproducible randomness from configured seeds; it does not establish unpredictability against an adversary that knows those seeds. Deployment would require server-secret randomness and an evaluation of defense-aware response strategies. Even secret randomness alone would not establish indistinguishability of traps from ordinary models.

D. Honest-Client Safety and Long-Term Accumulation

There is no fitted numerical z-score cutoff for statistical removal, but the policy is not parameter-free. Trap scale, histogram binning, bootstrap confidence, cohort support, confirmation count, and review/clearance lengths are design choices. A stable boundary is not a calibrated false-removal probability. At 100 clients the minimum statistical cohort support is ten; an isolated nonzero free rider can therefore escape this cohort gate.

Certificate memory deliberately has no fixed age limit. If an honest client's atypical geometry generates repeated strong evidence without two complete R cycles, it can eventually be removed. The profile gate's concentration alternative and persistent candidate populations make this a material long-horizon risk. The absence of honest removals through round 100 cannot exclude it. Longer honest-only and heterogeneous-population experiments are needed before making deployment safety claims.

E. Privacy, Collusion, and Availability

The protocol is not directly compatible with standard secure aggregation and does not provide differential privacy [9]. Avoiding client-reported loss and raw data access reduces required inputs but is not a privacy proof: identifiable returned models may still leak information. Colluding clients could share a genuinely trained response or infer the trap schedule, and Sybil identities could distort the assumed honest-majority reference. Neither attack is evaluated.

All active clients and complete A/B coverage are required for a certifying cycle. Missing/nonfinite measurements cannot certify, and the current simulator aborts on a nonfinite client response rather than implementing a complete availability defense. Dropout tolerance, malicious-message handling, restart integrity, authentication, and denial-of-service resistance require additional engineering and experiments. Checkpoint recovery preserves computation and evidence state after interruption; it does not create new evidence of honesty.

F. Computational and Communication Exposure

The bounds in Section VIII expose quadratic-in-population leave-one-out work and transient bootstrap memory. A large population or model can make auditing expensive even without learned detector training. Deliberately withheld trap responses and repeated honest follow-up are an availability and utility cost, not merely an implementation detail. Communication bytes, time to target utility, energy, and isolated detector work remain unmeasured.

X. Conclusion

This study evaluates a lightweight, server-observable challenge approach to free-rider screening, with strict norm separation, component-specific evidence memory, and bounded review. Across 32 completed MNIST/CIFAR-10 runs, all 1,120 free-rider client-cases are removed with no permanent honest removal. Most conditions complete by round 30; non-IID CIFAR-10 history averaging requires rounds 60 and 80. The result shows useful separation under the evaluated training and attack configurations, while the candidate and quarantine results show why final recall and precision alone are insufficient.

The principal next step is validation, not further adjustment to this seed: freeze the policy, evaluate unseen seeds and honest-only populations, and test adaptive/mixed attacks and longer horizons. Matched ablations and detector-only profiling should distinguish the contributions of traps, geometric gates, and memory from ordinary training and simulator costs. The method is a promising empirical screening mechanism, not a universal verifier of local computation.

Appendix A
Complete Per-Run Results

Tables X and XI retain every evaluated attack/partition/share combination. Machine-readable exports include per-run statistics, grouped summaries, full-cycle certificate histories, and per-round curves. No failed run is omitted or converted into a blank value.

Appendix B
Supporting Trajectories and Score Geometry

Figure 10 retains all cumulative detection trajectories, including the delayed non-IID CIFAR-10 FR3 cases. Figure 11 restores the full training horizon and shared 0–1 scale for interpreting the main-text accuracy zooms. Figures 12 and 13 show the complementary 30% client clouds using the same paired-score definition and monotone display transformation.

Appendix C
Reproducibility and Interpretation Notes

The source inputs are four batches under results/. Their names combine v15_4_mnist or v15_4_cifar10 with _seed42_fr30 or _seed42_fr40. The report builder validates completion markers, the 32-case grid, complete round logs, initial A/B coverage, fixed evaluation labels, removal uniqueness/timing, honest exclusion counts, and matching recorded detector policies and six implementation hashes. Input hashes are exported in generated/provenance.json; the six recorded source files are copied to evidence/source/. This is scoped provenance, not a claim

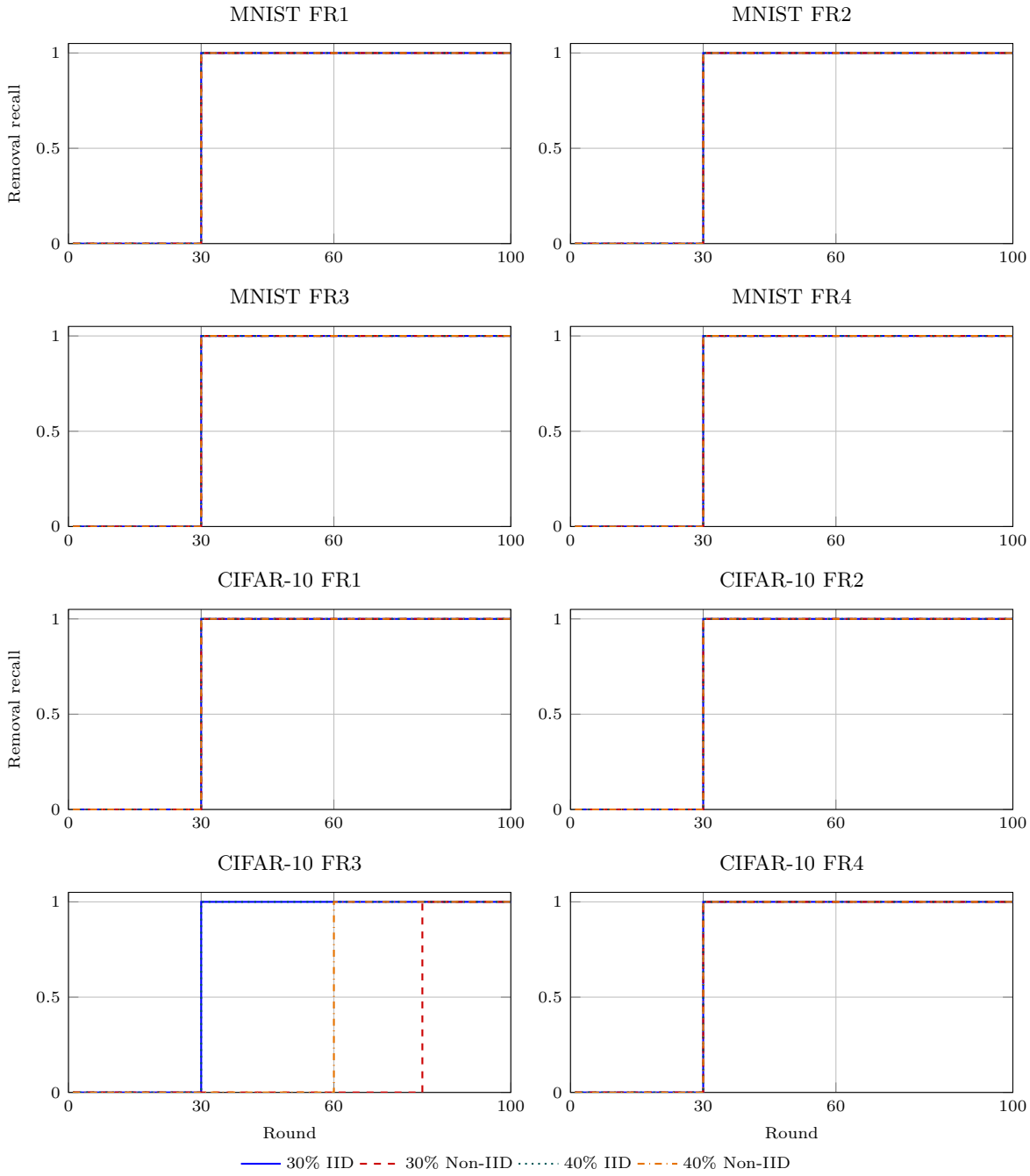


Fig. 10. Cumulative free-rider removal recall in all 32 runs. Enlarged two-column panels identify dataset and attack in each title; the four lines distinguish partition and attacker share. Coincident curves overlap. Complete outcomes are retained, including incomplete detection if present.

TABLE X
Complete per-run MNIST outcomes. Every run has 100 rounds and seed 42. Definitions match Tables I and II; all outcomes, including failures if present, are retained.

FR (%)	Partition	Attack	TP	FP	FN	t_{all}	Accuracy (%)	C_H	Q_H	q_H
30	IID	FR1	30	0	0	30	99.02	1	0	0
30	IID	FR2	30	0	0	30	99.00	5	0	0
30	IID	FR3	30	0	0	30	99.00	8	0	0
30	IID	FR4	30	0	0	30	98.82	2	0	0
30	Non-IID	FR1	30	0	0	30	99.01	7	0	0
30	Non-IID	FR2	30	0	0	30	99.00	6	0	0
30	Non-IID	FR3	30	0	0	30	98.98	8	0	0
30	Non-IID	FR4	30	0	0	30	98.96	6	0	0
40	IID	FR1	40	0	0	30	98.90	1	0	0
40	IID	FR2	40	0	0	30	98.91	5	0	0
40	IID	FR3	40	0	0	30	98.85	5	0	0
40	IID	FR4	40	0	0	30	98.77	5	0	0
40	Non-IID	FR1	40	0	0	30	98.96	2	0	0
40	Non-IID	FR2	40	0	0	30	98.96	7	0	0
40	Non-IID	FR3	40	0	0	30	98.93	34	1	20
40	Non-IID	FR4	40	0	0	30	98.98	12	0	0

TABLE XI
Complete per-run CIFAR-10 outcomes. Every run has 100 rounds and seed 42. Definitions match Tables I and II; all outcomes, including failures if present, are retained.

FR (%)	Partition	Attack	TP	FP	FN	t_{all}	Accuracy (%)	C_H	Q_H	q_H
30	IID	FR1	30	0	0	30	78.07	2	0	0
30	IID	FR2	30	0	0	30	77.93	0	0	0
30	IID	FR3	30	0	0	30	78.17	2	0	0
30	IID	FR4	30	0	0	30	78.03	7	0	0
30	Non-IID	FR1	30	0	0	30	75.80	8	0	0
30	Non-IID	FR2	30	0	0	30	75.58	2	0	0
30	Non-IID	FR3	30	0	0	80	75.36	70	1	20
30	Non-IID	FR4	30	0	0	30	75.78	7	0	0
40	IID	FR1	40	0	0	30	77.46	0	0	0
40	IID	FR2	40	0	0	30	77.65	0	0	0
40	IID	FR3	40	0	0	30	77.27	4	0	0
40	IID	FR4	40	0	0	30	77.36	8	0	0
40	Non-IID	FR1	40	0	0	30	75.28	9	0	0
40	Non-IID	FR2	40	0	0	30	75.43	4	0	0
40	Non-IID	FR3	40	0	0	60	74.89	52	9	180
40	Non-IID	FR4	40	0	0	30	75.14	7	0	0

that the complete historical environment or every helper file was frozen. The score exports retain raw paired coordinates and per-class means, medians, quartiles, finite counts and undefined counts for every complete evidence cycle. Additional resource-comparison inputs and the architecture hash are recorded in generated/visual_provenance.json. The historical v15.3 cost table is separate context and is never pooled with v15.4 detection totals.

Rebuild from the repository root by invoking build_paper_v6_full_study.py with Python and the –compile flag. The builder reads experiment inputs and writes only this new paper version. Previous paper versions and all experiment logs remain unchanged. The older v2/v8 results are not pooled with the current method: in particular, FR1 semantics, confirmation, reference construction, and screening policy differ.

Interrupted runs resume their last published checkpoint with loader/RNG state, attack histories, selected probe positions, quarantine and confirmation memory. The runtime table reports logged execution measures rather than powered-off calendar waiting. None of the reported resource values should be relabelled as detector-only overhead or interpreted as evidence of privacy/security guarantees.

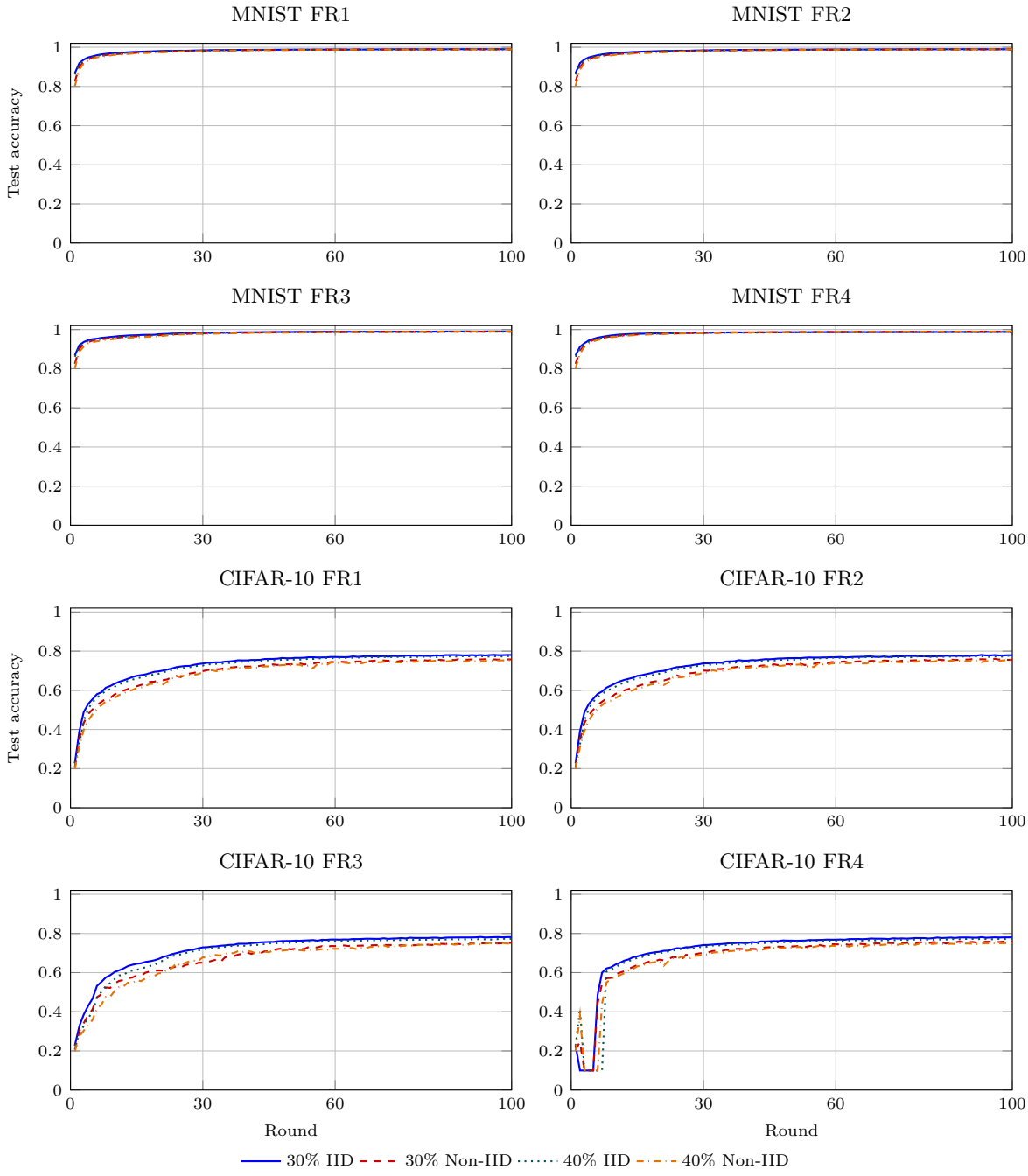


Fig. 11. Full, unzoomed global test-accuracy trajectories for all 32 runs, providing context for the late-training detail in the main text.

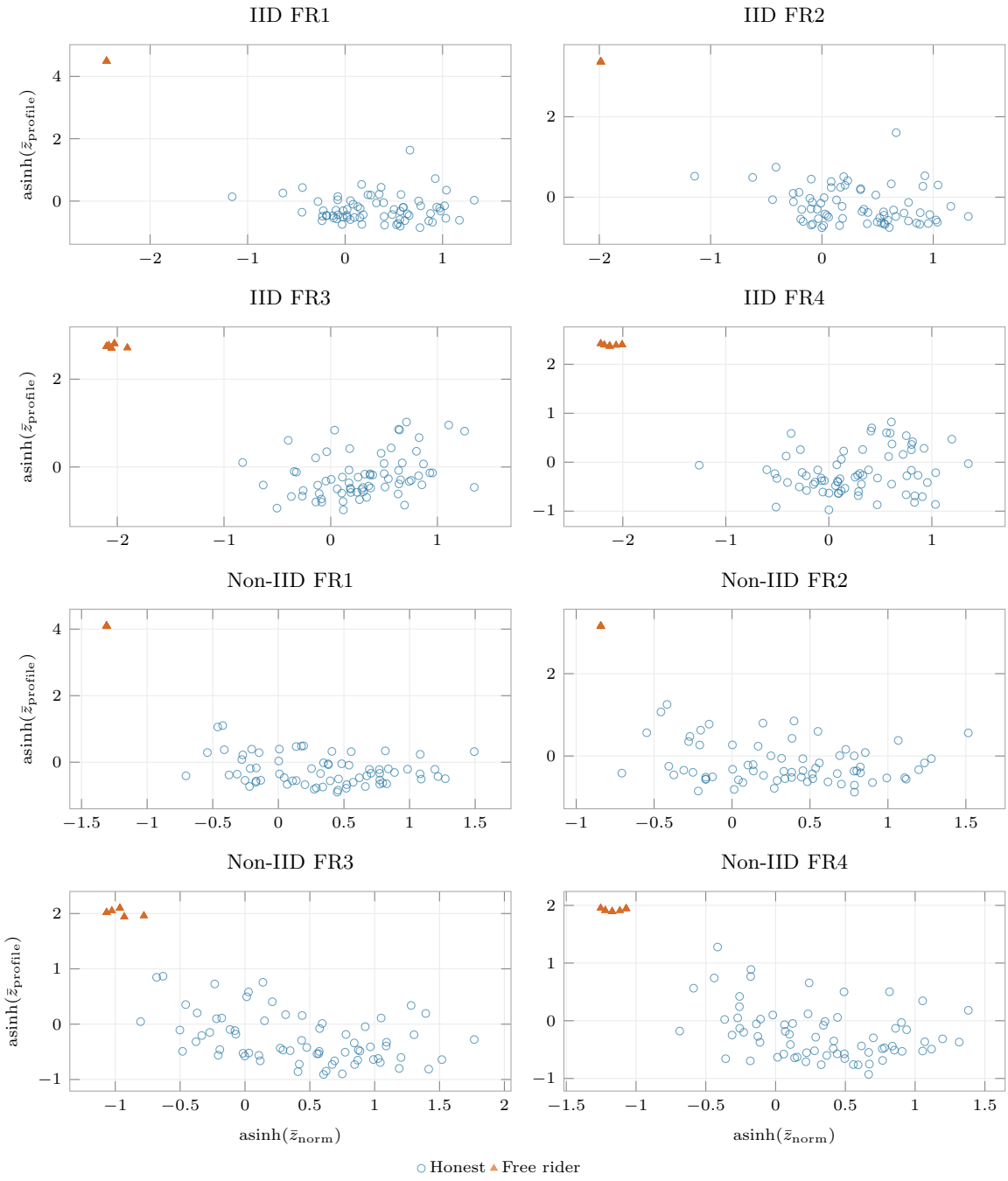


Fig. 12. MNIST: round-20 score geometry at 30% free riders (all eight settings). One point is one client, averaging the A/B norm-z and profile-z scores separately. The monotone asinh display compresses extreme tails without clipping; zero remains zero, but distances are transformed. Panels autoscale independently. Labels color evaluation only, never references or decisions. 0 points have undefined coordinates and are omitted; coincident points overlap. No plotted line is a removal cutoff.

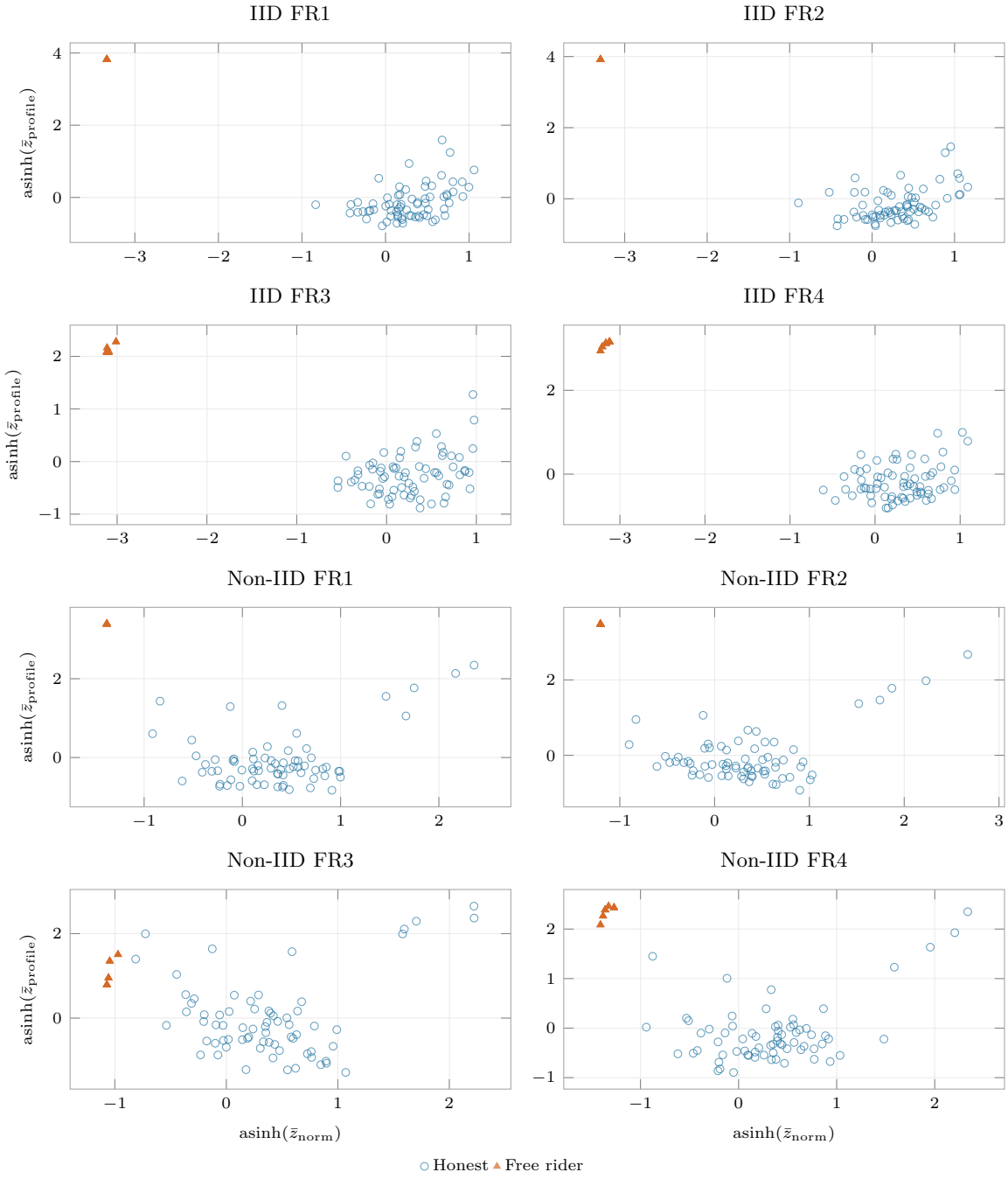


Fig. 13. CIFAR-10: round-20 score geometry at 30% free riders (all eight settings). One point is one client, averaging the A/B norm-z and profile-z scores separately. The monotone asinh display compresses extreme tails without clipping; zero remains zero, but distances are transformed. Panels autoscale independently. Labels color evaluation only, never references or decisions. 0 points have undefined coordinates and are omitted; coincident points overlap. No plotted line is a removal cutoff.

References

[1] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, "Communication-efficient learning of deep networks from decentralized data," in Proceedings of AISTATS, vol. 54, 2017, pp. 1273–1282. [Online]. Available: https://proceedings.mlr.press/v54/mcmahan17a.html

[2] J. Lin, M. Du, and J. Liu, "Free-riders in federated learning: Attacks and defenses," arXiv:1911.12560, 2019. [Online]. Available: https://arxiv.org/abs/1911.12560

[3] Y. Fraboni, R. Vidal, and M. Lorenzi, "Free-rider attacks on model aggregation in federated learning," in Proceedings of AISTATS, vol. 130, 2021, pp. 1846–1854. [Online]. Available: https://proceedings.mlr.press/v130/fraboni21a.html

[4] E. Bagdasaryan, A. Veit, Y. Hua, D. Estrin, and V. Shmatikov, "How to backdoor federated learning," in Proceedings of the Twenty Third International Conference on Artificial Intelligence and Statistics, ser. Proceedings of Machine Learning Research, vol. 108, 2020, pp. 2938–2948. [Online]. Available: https://proceedings.mlr.press/v108/bagdasaryan20a.html

[5] Z. Zhu, J. Shu, X. Zou, and X. Jia, "Advanced free-rider attacks in federated learning," in NeurIPS Workshop on New Frontiers in Federated Learning, 2021. [Online]. Available: https://neurips2021workshopfl.github.io/NFFL-2021/papers/2021/Zhu2021.pdf

[6] B. Wang, H. Li, X. Liu, and Y. Guo, "FRAD: Free-rider attacks detection mechanism for federated learning in AIoT," IEEE Internet of Things Journal, vol. 11, no. 3, pp. 4377–4388, 2024.

[7] P. G. Recasens, Á. Horváth, A. Gutierrez-Torre, J. Torres, J. L. Berral, and B. Pejó, "FRIDA: Free-rider detection using privacy attacks," Journal of Information Security and Applications, vol. 97, p. 104357, 2026. [Online]. Available: https://arxiv.org/abs/2410.05020

[8] J. Wang, X. Chang, J. Mišić, V. B. Mišić, and Y. Wang, "PASS: A parameter audit-based secure and fair federated learning scheme against free-rider attack," IEEE Internet of Things Journal, vol. 11, no. 1, pp. 1374–1384, 2024. [Online]. Available: https://arxiv.org/abs/2207.07292

[9] K. Bonawitz, V. Ivanov, B. Kreuter, A. Marcedone, H. B. McMahan, S. Patel, D. Ramage, A. Segal, and K. Seth, "Practical secure aggregation for privacy-preserving machine learning," in Proceedings of ACM CCS, 2017, pp. 1175–1191. [Online]. Available: https://research.google/pubs/practical-secure-aggregation-for-privacy-preserving-machine-learning/

[10] N. Otsu, "A threshold selection method from gray-level histograms," IEEE Transactions on Systems, Man, and Cybernetics, vol. 9, no. 1, pp. 62–66, 1979.

[11] B. Efron, "Bootstrap methods: Another look at the jackknife," The Annals of Statistics, vol. 7, no. 1, pp. 1–26, 1979.

[12] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," Proceedings of the IEEE, vol. 86, no. 11, pp. 2278–2324, 1998.

[13] A. Krizhevsky, "Learning multiple layers of features from tiny images," University of Toronto, Tech. Rep., 2009. [Online]. Available: https://www.cs.toronto.edu/~kriz/cifar.html